Chapter 22

Threat Tree Patterns

In this chapter:

Spoofing an External Entity or a Process 271
Tampering with a Process 273
Tampering with a Data Flow 274
Tampering with a Data Store 276
Repudiation 278
Information Disclosure of a Process 280
Information Disclosure of a Data Flow 281
Information Disclosure of a Data Store 282
Denial of Service Against a Process 284
Denial of Service Against a Data Flow 285
Denial of Service Against a Data Store 286
Elevation of Privilege 287

In Chapter 9, "Stage 4: Risk Analysis," we mentioned threat trees that reflect common attack patterns and help application designers think about security conditions in the system. This chapter itemizes the threat tree patterns and discusses what you should think about when designing and testing an application.

Chapter 9 presented an important table—Table 9-5, "Mapping STRIDE to DFD Element Types"—that is repeated here. The table shows the threat types (STRIDE) that apply to each data flow diagram (DFD) element type.

DFD Element Type	S	T	R	I	D	E
External Entity	X		X			
Data Flow		X		X	X	
Data Store		X	†	X	X	
Process	X	X	X	X	X	X

The dagger mark (†) in this table indicates a specific kind of data store, notably a data store that records logging or auditing data. This kind of data store is subject to repudiation threats because an attacker might attempt to cover his tracks by modifying or erasing the data.

For each valid intersection in the table, a threat tree shows the possible security-related preconditions for that STRIDE category. The leaf nodes of each tree can aid in secure design and security testing.

In this chapter, we'll look at each tree as well as associated design and security questions. Note that some trees cascade. For example, the tree in Figure 22-1 shows the conditions that could lead to spoofing threats against an external entity or a process. The circle at the right in Figure 22-1 shows tampering threats against the authentication process. This means that someone can indirectly spoof, say, a user by attacking the process that determines whether the user is who she claims to be. In some scenarios, this is a valid attack path.

For every threat in your threat models, you should consult this chapter for the appropriate threat tree and its accompanying table—each threat tree is followed by a related table—to confirm that you have considered all the appropriate design and testing concepts.

Spoofing an External Entity or a Process

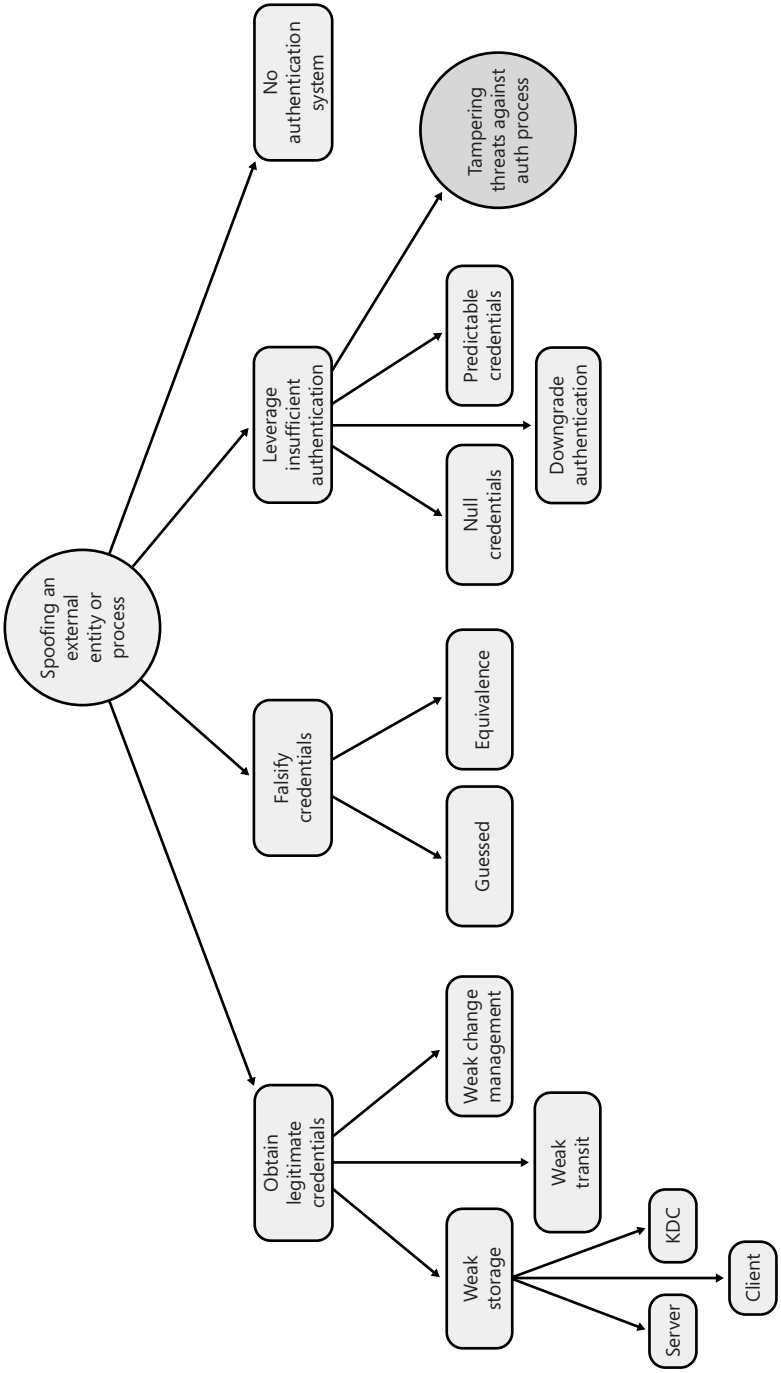

Figure 22-1 Threat tree for spoofing an external entity or a process.

Table 22-1 **Design and Test Considerations: Spoofing External Entity or Process**

Leaf Node	Design	Test
Weak server-credential storage	Are credentials held at the server? If yes, how are they protected?	Probe any place where credentials may be stored to see if the credentials are well protected. Determine how the information is encrypted and where the encryption keys are stored. Set a known credential and then scour storage for that credential.
Weak client-credential storage	Are credentials held at the client? If yes, how are they protected?	See above.
Weak key-distribution-center (KDC) storage	Do you have a key-distribution center? If yes, how are the credentials protected?	See above.
Weak credential transit	How are credentials transmitted between two endpoints?	Update or set a known credential and then listen on the wire for that credential.
Weak credential-change management	What is the protocol for updating a credential? How secure is it? Who reviewed it?	Set a known credential and then listen on the wire for that credential.
Guess credential	What is the credential complexity policy? Is it adequate? Is there an incorrect credential time-out?	Try brute-forcing credentials.
Credential equivalence	Could two or more credentials be treated to the same credential?	This is hard to test without using an exhaustive key-space attack.
Null credential	Does your application support a null credential or an account with no password?	Try making an anonymous connection or a connection with no password.
Downgrade authentication	Does your application support an older and less-secure authentication scheme? If yes, is the scheme enabled by default? If yes, why?	Build a client or server that negotiates with the older protocol and then revisit the spoofing threat tree using the downgraded scheme.

Table 22-1 **Design and Test Considerations: Spoofing External Entity or Process**

Leaf Node	Design	Test
Predictable credential	How random is the credential? A good example of a predictable credential is a cookie-based authentication scheme that uses incrementing values to identify users.	If the endpoint creates a credential, connect, look at the cookie, and then connect with another computer and look at the credential. Can you see a pattern in the credential?
No authentication scheme	Do you really need to forgo an authentication scheme?	Determine the amount of data an unauthenticated user could gather or which assets that user could access. Can that user access more than he or she should?

Tampering with a Process

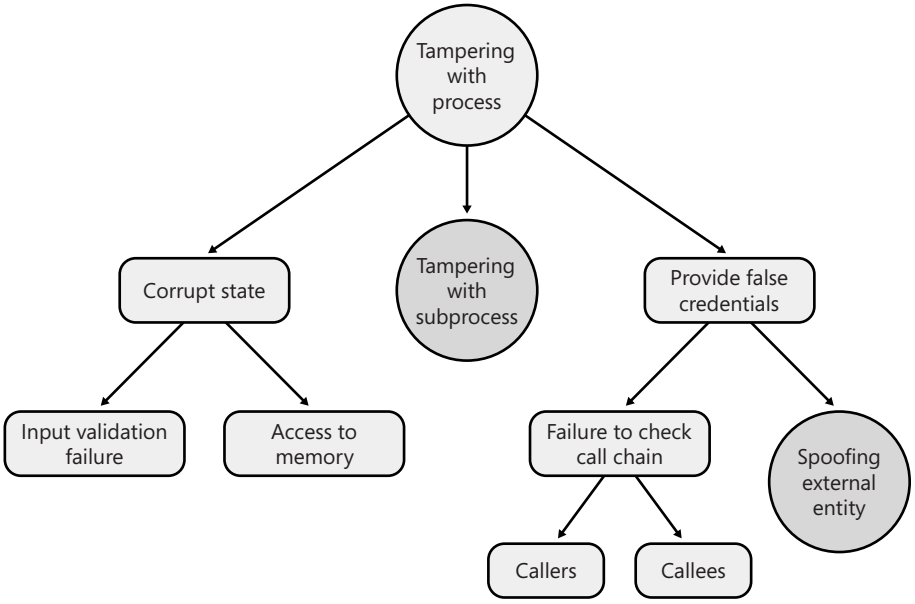

Figure 22-2 Threat tree for tampering with a process.

Table 22-2 **Design and Test Considerations: Tampering with a Process**

Leaf Node	Design	Test
Input validation failure	Is all input verified for correctness?	Fuzz testing is an effective testing technique for input validation issues.
Access to memory	This is hard to defend against, but for some processes, perhaps you need to defend against corruption of internal state. Examples of memory access issues include protecting against local administrators using debuggers. If you do not want to defend against this scenario, call the scenario out as an assumption in the threat model: you're not protecting your application from administrators with debuggers!	Using a debugger, determine what privileges are required to corrupt the internal application state. Again, this attack might be out of scope, so verify that it is out of scope in the threat model.
Callers	Do you trust code that calls your code? For example, do you have a LinkDemand (Microsoft 2005a) on your code that does not require the caller's caller to have the permissions you are link demanding?	Try mounting a luring attack (Brown 2001).
Callees	Do you trust the code you call? For example, your code may call code that uses CodeAccessPermission.Assert insecurely (Microsoft 2005b).	If the application has extensibility mechanisms, build a component that uses Assert to use a permission in an insecure manner.

Tampering with a Data Flow

You'll notice that the threat tree in Figure 22-3 references the message and the channel. The message is the data that travels across a channel, and you can opt to protect the message itself from attack or to use a protected channel. For example, you could decide to use an application-level digital signature on the message to protect the message, or you could use SSL/TLS (Secure Sockets Layer/Transport Layer Security) to provide integrity protection through the use of message authentication codes.

Finally, there is some philosophical debate in the author's team about whether tampering with a data flow requires you to tamper with the message *and* the channel or the message *or* the channel. For the moment, we're leaving it as an "or" condition.

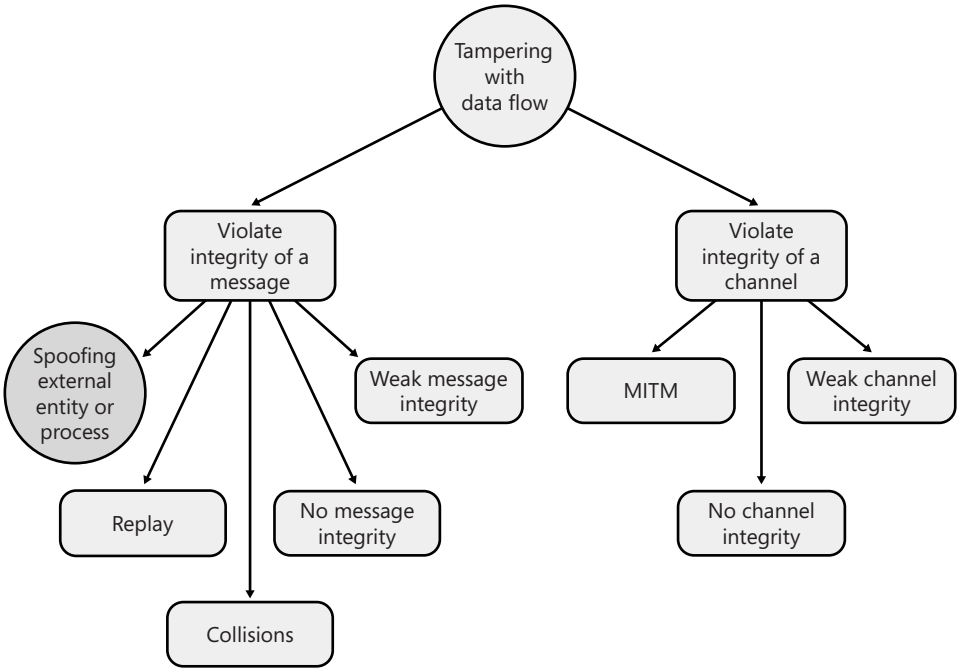

Figure 22-3 Threat tree for tampering with a data flow.

Table 22-3 Design and Test Considerations: Tampering with a Data Flow

Leaf Node	Design	Test
Replay	Is the dataflow defended (hashed, MAC'd, or signed) using antireplay defenses such as time stamps or counters?	Replay valid messages.
Collisions	See above.	See above.
No message integrity	Why is there no message integrity? Does this component's DFD include an unmitigated threat that should be fixed?	Fuzz the messages.
Weak message integrity	How good is the message-integrity algorithm, technology, or protocol? Is it appropriate?	See above.
Violated channel through man-in-the-middle (MITM)	Is the channel protected by an appropriate integrity technology?	Fuzz the channel.
No channel integrity	Why is there no channel integrity? Does this component's DFD include an unmitigated threat that should be fixed?	See above.
Weak channel integrity	How good is the channel-integrity algorithm, technology, or protocol? Is it appropriate?	See above.

Tampering with a Data Store

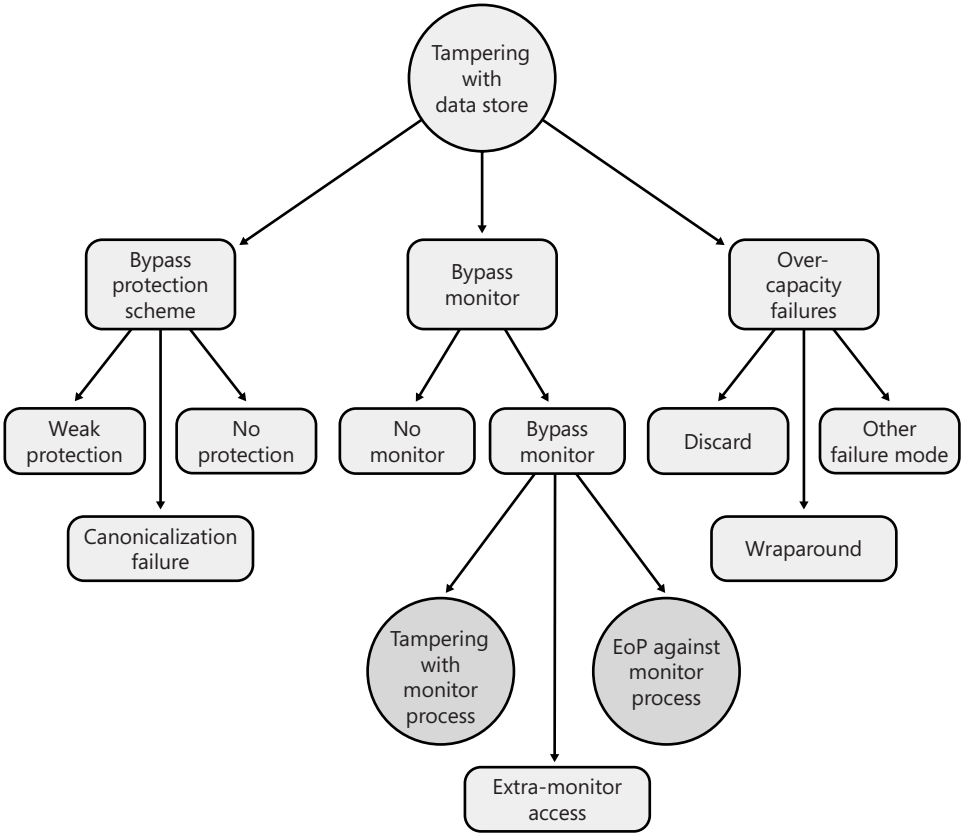

Figure 22-4 Threat tree for tampering with a data store.

Table 22-4 Design and Test Considerations: Tampering with a Data Store

Leaf Node	Design	Test
No protection	Why is there no protection scheme? Is this by design?	There is nothing to test.
Canonicalization failure	Does any code rely on a name, such as a file name, to determine access? If yes, make sure the code looks for only valid names and does not filter out illegal names.	Try malforming names, trailing dots, trailing spaces, device names, and so on (Howard, LeBlanc, and Viega 2005).

Table 22-4 **Design and Test Considerations: Tampering with a Data Store**

Leaf Node	Design	Test
Weak protection	Look at the permissions on all objects to determine whether they offer the correct level of protection.	Determine whether compromising the data store as a subject associated with the permission set could lead to application failure. You have a problem if the application consuming the data store fails and has higher privilege than the subject that is able to tamper with the data store.
No monitor	Is the data store on a system that does not support access checks or semantic checks? An example of a semantically different data is numeric versus alphabetic data; they are both made of ASCII or Unicode characters, but they each have a different meaning.	For semantic checks, test to determine whether type checks exist.
Extra-monitor access	Is all access to the data store throttled through one entry point? Is all access totally mediated?	Can you access the data store without following the process governing access? For example, if an application has logic to grant or deny access to the store, can you access the data store through other means, such as FTP or file sharing?
Discard	When the data store is full, is data discarded? If so, why?	Observe what happens when the data store is flooded.
Wraparound	When the data store is full, is data written to the beginning of the data store? If so, why?	See above.
Other failure mode	When the data store is full, is data dropped and not written to the store? Does the application crash? In some cases, a crash may be appropriate if a critical security log is full.	See above.

Repudiation

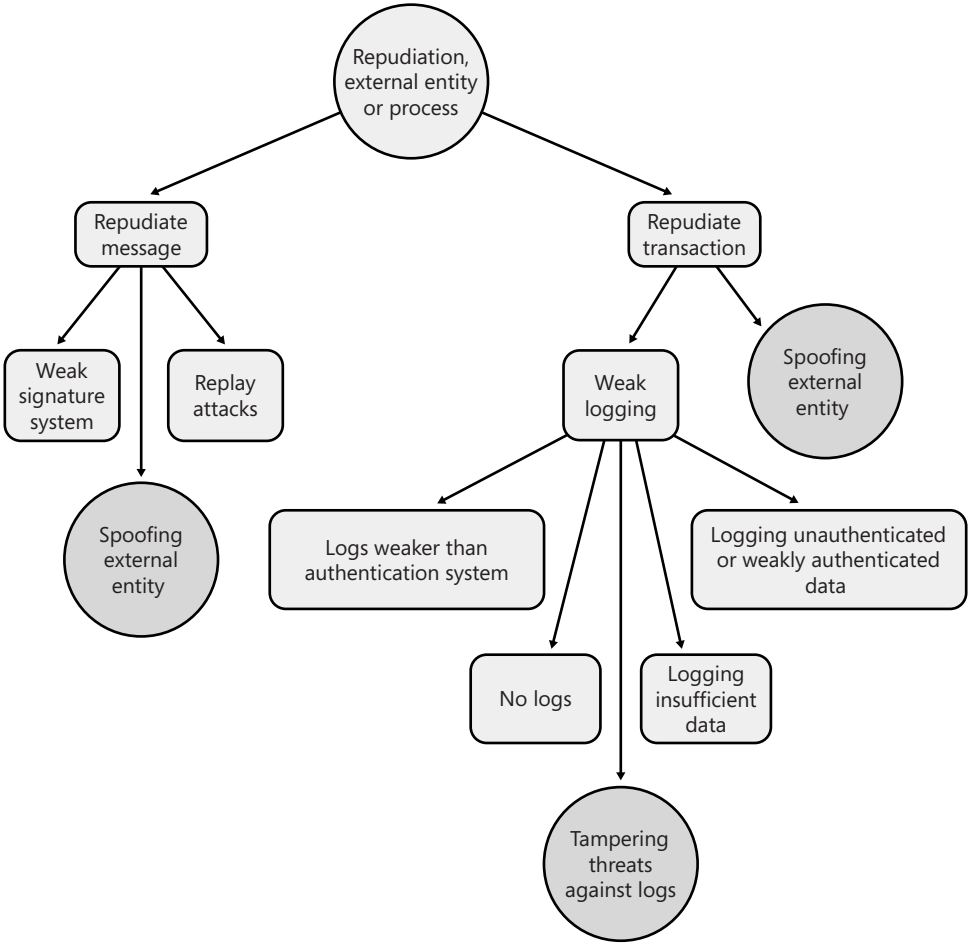

Figure 22-5 Repudiation threat tree.

Table 22-5 **Design and Test Considerations: Repudiation**

Leaf Node	Design	Test
Weak signature system	If you are using digital signatures, is the system strong? Consider having a security expert or cryptographer review the design. Where are the private keys stored, and how well protected are they?	Can you access the private keys associated with the signatures? Make sure each signature covers every bit of data you intend it to, and make sure that the signature is invalid when any bit of data is changed. For an example of this kind of failure: (GNU Privacy Guard 2006).
Replay attacks	See "Replay" in Table 22-3.	See "Replay" in Table 22-3.
No logs	Why are there no logs?	There is nothing to test.
Logs weaker than the authentication system	The logging system should be written to only by trusted code, and less-trusted users should be unable to write directly to the log files.	Try writing to the log files directly.
Logging unauthenticated or weakly authenticated data	Data written to logs should be reliable and generated by the application performing the logging.	Try generating requests of the application performing logging to determine whether you can write arbitrary data that might make it difficult to understand the data. A good example of this kind of issue is found in Apache 1.3.x (CVE 2003).
Logging insufficient data	It's important to log appropriate data to aid in supporting repudiation claims. Are you logging enough data? You might want to talk to a computer-forensics expert or computer-auditing expert to determine an appropriate level of logging.	There is nothing to test.

Information Disclosure of a Process

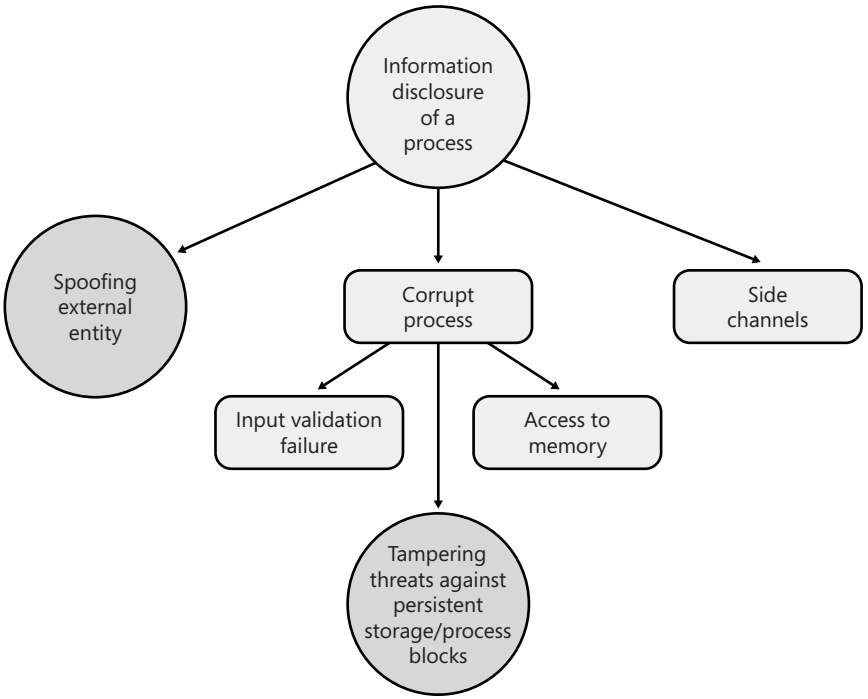

Figure 22-6 Threat tree for information disclosure of a process.

Table 22-6 Design and Test Considerations: Information Disclosure of a Process

Leaf Node	Design	Test
Input validation failure	See "Input validation failure" in Table 22-2.	See "Input validation failure" in Table 22-2.
Access to memory	See "Access to memory" in Table 22-2.	See "Access to memory" in Table 22-2.
Side channels	Could any side channels (Wikipedia 2006) disclose data? An example of a side channel is the existence of a special file, which means the process handles certain sensitive data. For a discussion of other examples of side channels: (CERT 2001, Anley 2002, CERT 2002a, CERT 2003, Lucas 2005, SecuriTeam 2005, Mimoso 2006).	Side channels can be hard to test for; a best practice is to review past side-channel exploitations (see the list in the Design column in this table) to determine whether they could occur in your software.

Information Disclosure of a Data Flow

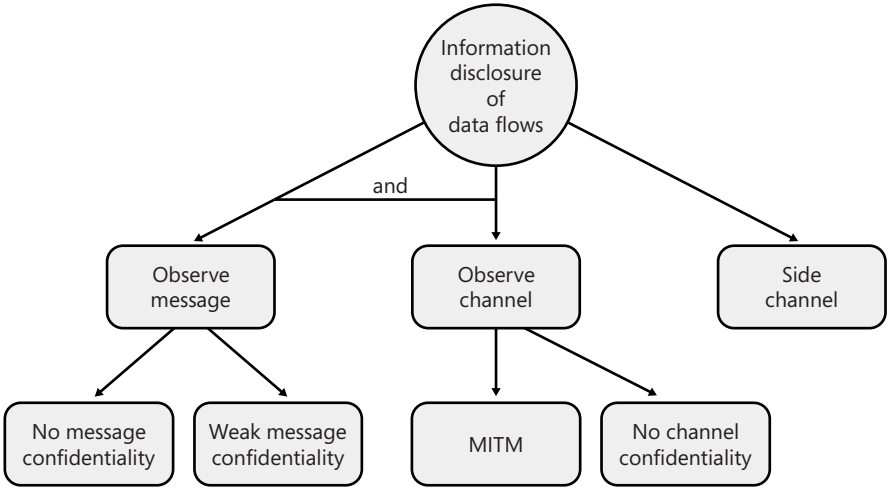

Figure 22-7 Threat tree information disclosure of a data flow.

Table 22-7 Design and Test Considerations: Information Disclosure of a Data Flow

Leaf node	Design	Test
No message confidentiality	Why is there no message confidentiality? What is the impact if the data is disclosed?	Can you view any sensitive data that should be protected?
Weak message confidentiality	Is the data well defended? What cryptographic algorithms are used, and where are the keys stored?	Can you access the keys, or is the encryption really "encraption"?
Channel observable through man-in-the-middle (MITM)	If a rogue listener can be placed between two endpoints on the data flow, can the listener read the data?	Build a man-in-the-middle tool, perhaps a proxy, to listen and potentially read the data.
No channel confidentiality	Why is there no channel confidentiality? What is the impact if the data is disclosed?	Can you view sensitive data that should be protected?
Side channels	See "Side channels" in Table 22-6.	See "Side channels" in Table 22-6.

Information Disclosure of a Data Store

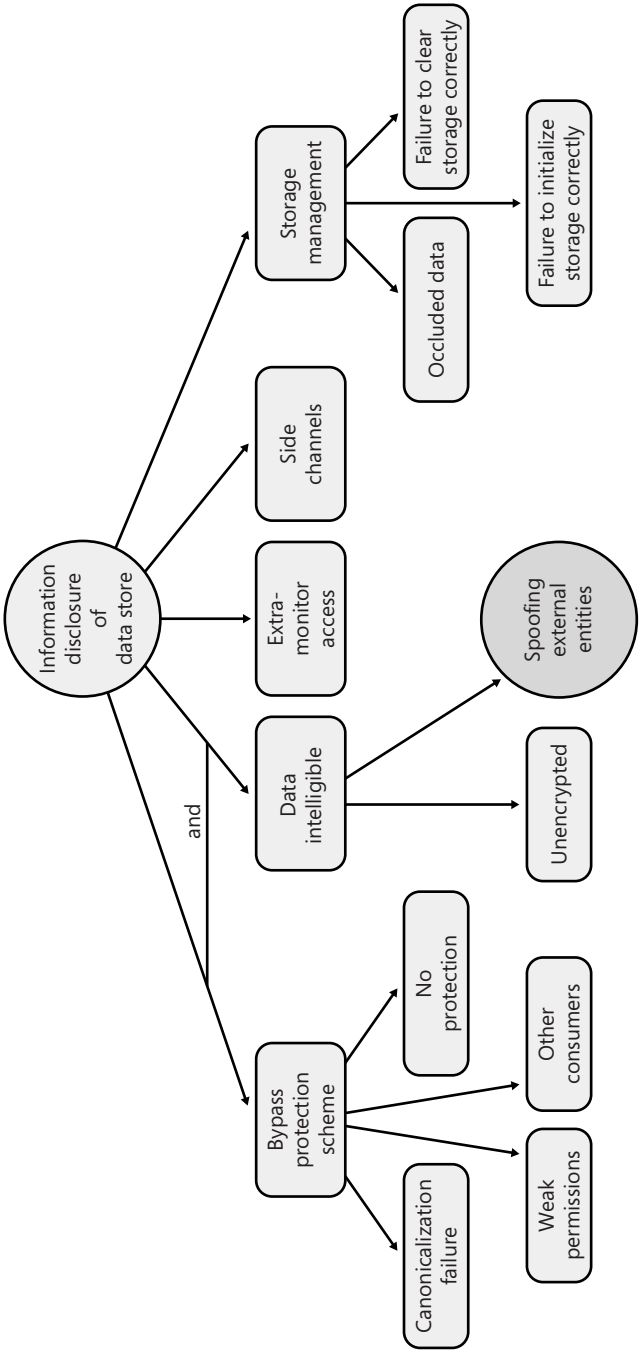

Figure 22-8 Threat tree for information disclosure of a data store.

Table 22-8 **Design and Test Considerations: Information Disclosure of a Data Store**

Leaf Node	Design	Test
Canonicalization failure	See "Canonicalization failure" in Table 22-4.	See "Canonicalization failure" in Table 22-4.
Weak protection	See "Weak protection" in Table 22-4.	See "Weak protection" in Table 22-4.
Other consumers	Are there data-store consumers other than this application (for example, indexing or searching applications)?	Can you use other methods (such as search engines) to reveal the protected data?
No protection	See "No protection" in Table 22-4.	See "No protection" in Table 22-4.
No encryption	If the data is not encrypted, are other protections good enough?	Is the data viewable in the data store when accessed using a simple hex-dump tool?
Extra-monitor access	See "Extramonitor access" in Table 22-4.	See "Extramonitor access" in Table 22-4.
Side channels	See "Side channels" in Table 22-6.	See "Side channels" in Table 22-6.
Occluded data	Does your application support undo or recovery operations? Does it support object properties that might contain sensitive data? If so, how will you erase this data? For a good reference for dealing with these issues: (National Security Agency 2006).	If the application supports "hiding" data, enter some known text, hide the data, and then search the file on disk for the data. Although the data might not be visible in the application, it might be visible when you perform a binary scan.
Failure to initialize storage correctly	Is storage in the data store set to a known value before use? If not, why? Setting data to a known value is rarely a performance issue.	If possible, set a known data value in the uninitialized data store. Then when the application runs, find out if you can still read the data.
Failure to clear storage correctly	Is storage in the data store set to a known value after use? If not, why? Setting data to a known value is rarely a performance issue.	Set data to a known value. When the application has finished using the data, scour the data store to see if the data is still there. For interesting examples of failure to scrub data correctly: (Howard 2002, Chow et al. 2005).

Note Some information disclosure threats can be privacy issues if the data being disclosed is private or personally identifiable information (PII). Refer to Chapter 8, "Stage 3: Product Risk Assessment," for more information on the subject.

Denial of Service Against a Process

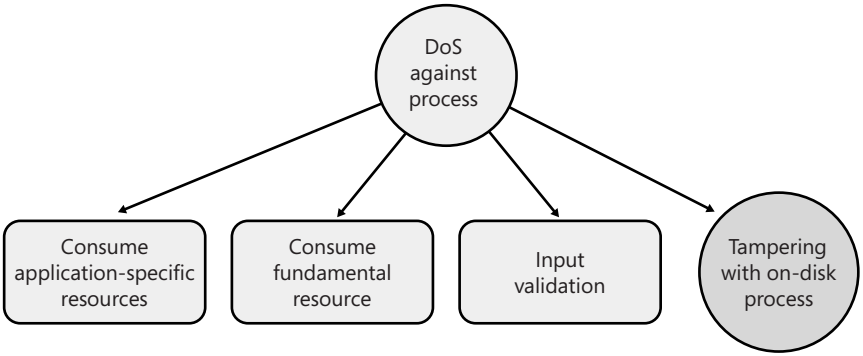

Figure 22-9 Threat tree for denial of service against a process.

Table 22-9 Design and Test Considerations: Denial of Service Against a Process

Leaf Node	Design	Test
Application-specific resource consumption	Are there limits on application-specific resource consumption? Can unprivileged users consume large quantities of such resources? For a good example of malicious resource consumption: (CERT 2002b).	Identify all application-specific resources and determine what privilege level is required to consume the resources. There are issues if low-trust or anonymous users can consume large quantities of application-specific resources.
Fundamental resource consumption	Are there limits on fundamental resource consumption, such as memory, CPU, network bandwidth, or disk space? Can unprivileged users consume large quantities of such resources?	Attempt to consume large quantities of fundamental resources as an untrusted user.
Input validation	Is all input validated for correctness? Input validation is a broader variation of the canonicalization issues noted previously, because even valid input could trigger infinite recursions or cause the application to consume large amounts of CPU time.	Do intelligent fuzz testing.

Denial of Service Against a Data Flow

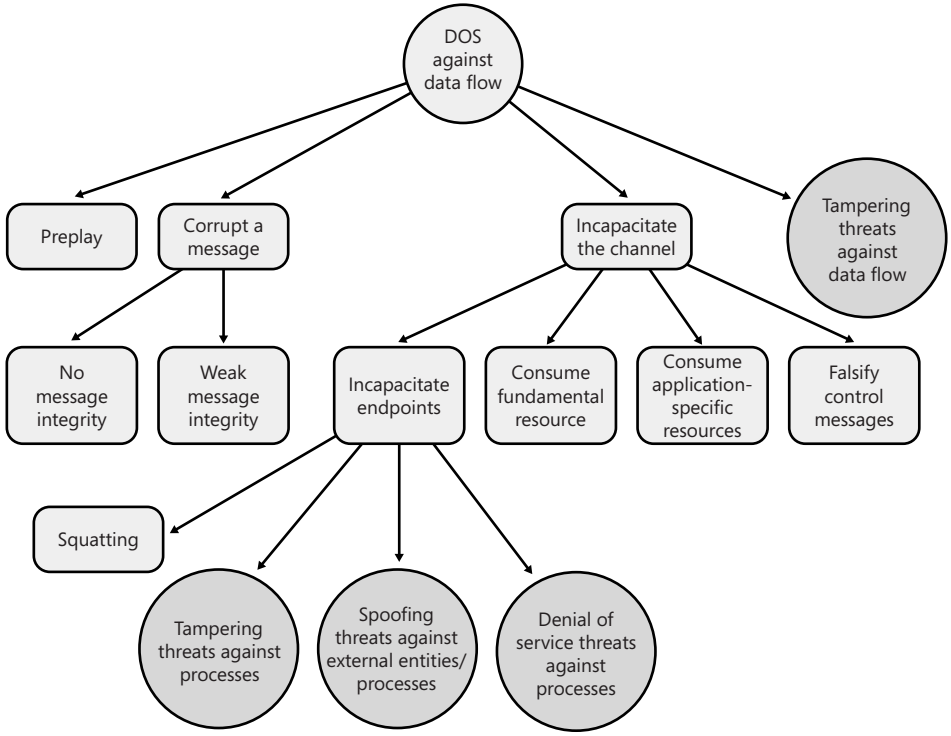

Figure 22-10 Threat tree for denial of service against a data flow.

Table 22-10 Design and Test Considerations: Denial of Service Against a Data Flow

Leaf Node	Design	Test
Preplay	An adversary makes a preplay attack when she performs an action before the valid application does so. How does your application respond to such attacks? How do you ensure that the action is performed by a valid user and not an attacker?	Build tests that perform partial communication. Then have another computer or process inject the continuation of the protocol.
No message integrity	See "No message integrity" in Table 22-3.	See "No message integrity" in Table 22-3.
Weak message integrity	See "Weak message integrity" in Table 22-3.	See "Weak message integrity" in Table 22-3.
Squatting	Can the data flow name (pipe name, for example) be hijacked or squatted?	Get a list of all the data flow names used by the application, create all these names ahead of time, and then start the application to see if the data flows are valid.

Table 22-10 Design and Test Considerations: Denial of Service Against a Data Flow

Leaf Node	Design	Test
Fundamental resources consumption	See "Fundamental resources consumption" in Table 22-9.	See "Fundamental resources consumption" in Table 22-9.
Application-specific resources consumption	See "Application-specific resources consumption" in Table 22-9.	See "Application-specific resources consumption" in Table 22-9.
Falsified control messages	Does the application use any messages to control the flow of data across the data flow? Is there a state-machine or transition diagram (Young 2000) used to determine the correct flow order? If so, how well is the state-machine followed?	Get a list of control messages used by the application protocols and then fuzz them. Don't fuzz the message data; just fuzz the control messages, or create out-of-order messages or incorrect messages. Also, if there is a state-machine, try to build messages that violate the normal transition flow.

Denial of Service Against a Data Store

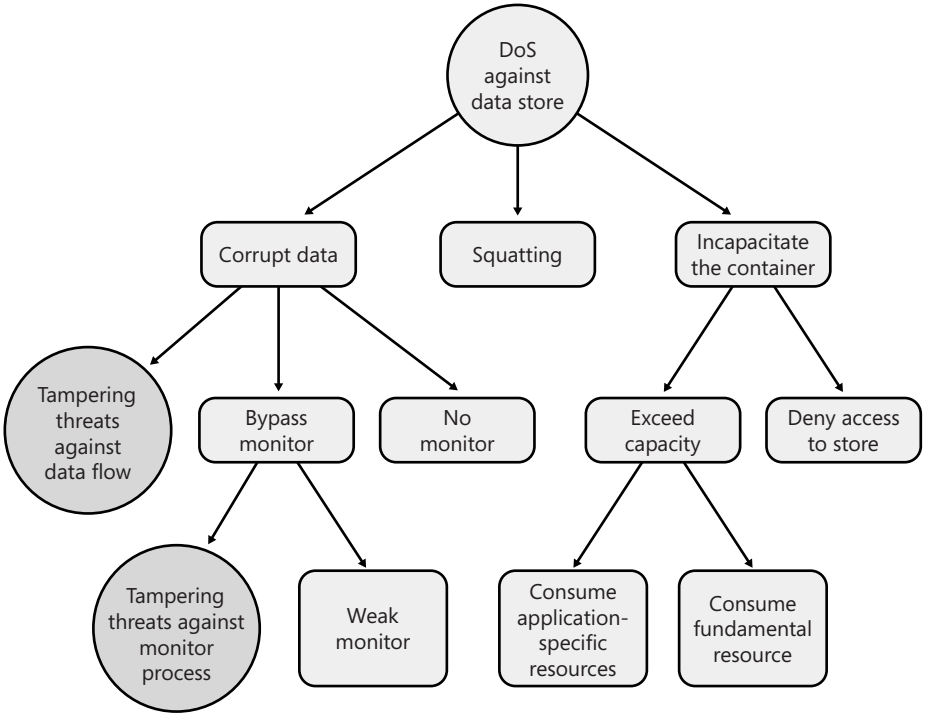

Figure 22-11 Threat tree for denial of service against a data store.

Table 22-11 **Design and Test Considerations: Denial of Service Against a Data Store**

Leaf Node	Design	Test
Weak protection	See "Weak protection" in Table 22-4.	See "Weak protection" in Table 22-4.
No protection	See "No monitor" in Table 22-4.	See "No monitor" in Table 22-4.
Squatting	See "Squatting" in Table 22-12.	See "Squatting" in Table 22-12.
Fundamental resources consumption	See "Fundamental resources consumption" in Table 22-9.	See "Fundamental resources consumption" in Table 22-9.
Application-specific resources consumption	See "Application-specific resources consumption" in Table 22-9.	See "Application-specific resources consumption" in Table 22-9.
Deny access to store	What happens if the application is denied access to the data store? Is there a permission on the object that allows untrusted users to deny access to or lock the file?	Try denying access to the file (for example, in Microsoft Windows, by setting an Everyone Deny-All ACE to the data store), or try locking the data source.

Elevation of Privilege

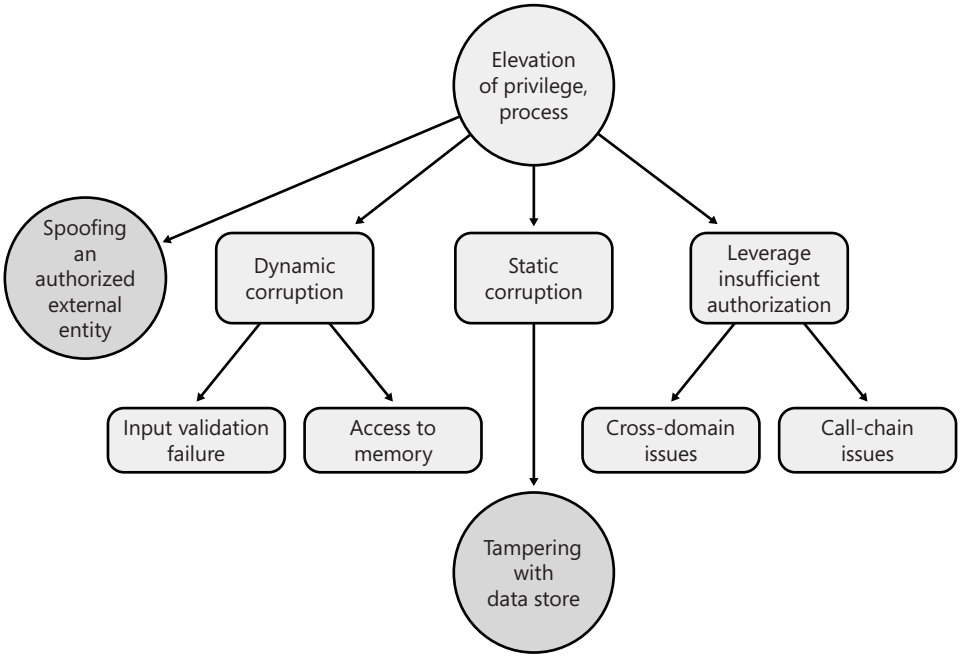

Figure 22-12 Threat tree for elevation of privilege.

Table 22-12 **Design and Test Considerations: Elevation of Privilege**

Leaf Node	Design	Test
Input validation failure	See "Input validation failure" in Table 22-2.	See "Input validation failure" in Table 22-2.
Access to memory	See "Access to memory" in Table 22-2.	See "Access to memory" in Table 22-2.
Cross-domain issues	Applications that host potentially malicious script from the Internet enforce a security model, often called the cross-domain security model or same-origin policy. All Web browsers do this, as do application frameworks like Java and Microsoft .NET Framework. The cross-domain security model prevents script and code that originated from one domain from interacting with content from another domain. The intention is to prevent script from a malicious Web site from having access to content from Hotmail.com in the security context of the user accessing Hotmail.com.	Can the application host multiple instances of a Web browser rendering engine? If so, can script running in one instance of the Web browser manipulate the other Web browser instance? If your application has an object model accessible to script running in a Web browser, how can malicious script from the Internet abuse the object model?
Call-chain issues	See "Callers" and "Callees" in Table 22-2.	See "Callers" and "Callees" in Table 22-2.

References

(Microsoft 2005a) .NET Framework Developer's Guide, "Demand vs. LinkDemand," *http://msdn2.microsoft.com/en-us/library/3ky50t49.aspx*. MSDN, 2005.

(Brown 2001) Brown, Keith. "Security in .NET: Enforce Code Access Rights with the Common Language Runtime," *http://msdn.microsoft.com/msdnmag/issues/01/02/cas/*. *MSDN Magazine*, February 2001.

(Microsoft 2005b) .NET Framework Developer's Guide, "Using the Assert Method," *http://msdn2.microsoft.com/en-us/library/91wteedy.aspx*. MSDN, 2005.

(Howard, LeBlanc, and Viega 2005) Howard, Michael, David LeBlanc, and John Viega. *19 Deadly Sins of Software Development*. New York, NY: McGraw-Hill, 2005. Chapter 15, "Improper File Access."

(GNU Privacy Guard 2006) Koch, Werner. "GnuPG does not detect injection of unsigned data," *http://lists.gnupg.org/pipermail/gnupg-announce/2006q1/000216.html*. March 2006.